



Docker在本地构建项目并部署到服务器

这份文档详细说明了如何从零开始，使用Docker在本地构建项目、打包Docker镜像，并将其部署到服务器。不涉及项目开发以及dockerfile编写，只涉及项目部署。

由于构建环境在内网服务器无法访问docker官方网站，需要本地构建镜像，以后上传镜像部署。

1. 准备工作 (Prerequisites)

在开始之前，请确保：

1. 本地环境：

- 已安装 [Docker](#)。 version>=25.10.17
- 可以构建docker本地环境（可以魔法上网（下载依赖））

2. 服务器环境：

- 已安装 Docker 和 Docker Compose。

3. 源代码：

- 项目代码结构组织逻辑如下
前后端分离项目（以数据结构项目为例）

```

web-project/          # 项目根目录
├─ web_data_structure/      # 前端子项目（Vue/React）
│   ├─ src/
│   ├─ Dockerfile
│   ├─ .dockerignore
│   └─ config/nginx.conf
├─ server_data_structure/    # 后端子项目（Spring Boot/Node.js/Flask/Django/Gin）
│   ├─ src/
│   ├─ Dockerfile
│   ├─ .dockerignore
│   └─ config/
├─ docker-compose.yml      # 核心：（必须）编排前端(Nginx)、后端、MySQL、Redis、Nginx
├─ .env                    # 全局环境变量（如数据库密码、服务端口）
├─ volumes/                # 数据持久化目录
│   ├─ mysql/              # MySQL数据挂载
│   ├─ redis/              # Redis数据挂载
│   ├─ backend-logs/       # 后端日志挂载
│   └─ frontend-static/    # 前端静态文件挂载（可选）
└─ sql/
    └─ ds-al.sql           # 数据库初始化脚本

```

单体项目

```

web-project/          # 项目根目录
├─ src/                # 业务源码目录（核心）
│   └─ ...（前端/后端源码，如vue/react/java/node代码）
├─ Dockerfile          # 核心（必选）：构建Docker镜像的指令文件
├─ .dockerignore       # 可选：排除不需要打包进镜像的文件（如node_modules、日志）
├─ docker-compose.yml  # 可选（推荐）：多容器编排（如web+数据库+缓存）
├─ .env                # 可选：环境变量配置（如数据库地址、端口）
├─ config/             # 可选：配置文件目录（如nginx.conf、app.conf）
├─ scripts/            # 可选：辅助脚本（如启动脚本、数据库初始化脚本）
├─ volumes/            # 可选：本地挂载目录（如日志、静态文件、数据持久化）
└─ README.md           # 可选：部署说明（如镜像构建/启动命令）

```

2. 本地构建镜像 (Build Images)

本地构建镜像需要魔法上网（或者添加国内镜像源）来下载所需要用到的依赖镜像，否则可能会构建失败。

介绍本地构建前后端分离项目镜像的步骤。

2.1 同时构建

1. 打开终端，进入项目根目录：

```
cd web-project
```

2. 执行构建命令（根据docker-compose.yml，会自动构建前端和后端）：

```
docker compose build
```

实例docker-compose.yml:

```

services:
  # 前端nginx项目
  frontend:
    build: ./web_data_structure
    image: ds-frontend:1.0
    ports:
      - "80:80"
    depends_on:
      - backend
    restart: always
    networks:
      - app-network

  # 后端java项目
  backend:
    build: ./server_data_structure
    image: ds-backend:1.0
    ## 设置了network可以避免端口冲突
    # ports: (可选 开放端口 )
    #   - "8080:8080"
    environment:
      - SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/ds_db
      - SPRING_REDIS_HOST=redis
      - SPRING_DATASOURCE_USERNAME=root
      - SPRING_DATASOURCE_PASSWORD={password}
    volumes:
      - ./volumes/backend-logs:/app/logs
    depends_on:
      - mysql
      - redis
    restart: always
    networks:
      - app-network

  # MySQL数据库服务（可选）
  mysql:
    image: mysql:8.0
    ## 设置了network可以避免端口冲突
    # ports:

```

```

#   - "3306:3306"
environment:
  - MYSQL_ROOT_PASSWORD={password}
  - MYSQL_DATABASE=ds_db
volumes:
  - ./volumes/mysql:/var/lib/mysql # 数据持久化
  - ./scripts/init-mysql.sql:/docker-entrypoint-initdb.d/init.sql # 初始化脚本
restart: always

# Redis缓存服务（可选）
redis:
  image: redis:7.0-alpine
  ## 设置了network可以避免端口冲突
  # ports:
  #   - "6379:6379"
  volumes:
    - ./volumes/redis:/data
  restart: always
  networks:
    - app-network

networks:
  app-network:
    driver: bridge

```

注意事项：

构建项目时的地址和端口需要和docker-compose.yml中的服务名称一致。

比如：

```

#前端
BACKEND_URL=http://localhost:8080
#后端
SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/ds_db # 连接MySQL容器
spring.redis.host=${SPRING_REDIS_HOST:localhost}

```



```
#前端
BACKEND_URL=http://backend:8080
#后端
SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/ds_db # 连接MySQL容器
spring.redis.host=${SPRING_REDIS_HOST:redis}
```

(如果是单体项目，只需要构建Dockerfile即可。本文档不再介绍单体项目的构建)

2.2 分开构建

分开构建的本质是将docker-compose.yml中的服务分开构建，分别为后端和前端构建 Docker 镜像。

2.2.1 构建后端镜像 (Backend)

1. 打开终端，进入后端目录：

```
cd server_data_structure
```

2. 执行构建命令（后端会自动编译代码）：

```
# ds-backend:1.0 ds-backend是镜像名称，1.0是版本号需要根据实际情况修改
docker build -t ds-backend:1.0 .
```

注意：这会下载 Maven 依赖并编译，第一次运行可能需要几分钟。

2.2.2 构建前端镜像 (Frontend)

1. 进入前端目录：

```
cd ../web_data_structure
```

2. 执行构建命令（使用生产环境配置）：

```
# ds-frontend:1.0 ds-frontend是镜像名称，1.0是版本号需要根据实际情况修改
docker build -t ds-frontend:1.0 .
```

3. 本地测试 (Local Test) - 可选

在上传服务器前，**建议本地测试成功以后再上传到服务器。**

注意：请确保docker-compose.yml中的服务名称和地址与docker-compose.yml中的服务名称一致。

0. 查看docker镜像

```
docker images
```

```
goose@goose:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
ds-backend:1.0	0964c9b7d540	1.06GB	410MB	
ds-frontend:1.0	30fb92c25198	102MB	29.8MB	
mysql:8.0	0275a35e79c6	1.08GB	247MB	

1. 回到 docker 根目录：

```
cd /web-project/
```

2. 启动服务：

```
docker-compose up -d
```

3. 访问测试：

- 前端页面：打开浏览器访问 `http://localhost:80`
- 如果一切正常，你应该能看到页面并能登录/获取数据。

4. 测试完成后停止服务：

```
docker-compose down
```

常见问题(FAQ)

- **端口冲突：**如果 80 被占用，请修改 `docker-compose.yml` 中的 `ports` 映射（例如 `8083:80`）。
- **数据库连接失败：**检查 `docker-compose.yml` 中后端服务的环境变量

`SPRING_DATASOURCE_URL` 是否指向了正确的数据库容器名（`db`）。

- **访问失败**：检查代码中前端API接口是否正确设置成后端服务的名称（`backend`） 需要重新构建。
- **其他问题**：检查 docker 日志。

4. 打包镜像 (Package Images)

如果服务器没有配置 Docker Registry（镜像仓库），我们可以将镜像保存为文件传输。

1. 导出后端镜像：

```
docker save -o backend.tar ds-backend:1.0
```

2. 导出前端镜像：

```
docker save -o frontend.tar ds-frontend:1.0
```

3. 导出mysql镜像：(如果需要)

```
docker save -o mysql.tar mysql:8.0
```

4. 导出redis镜像：(如果需要)

```
docker save -o redis.tar redis:7.0-alpine
```

此时目录下应该有四个大文件：`backend.tar`、`frontend.tar`、`mysql.tar`、`redis.tar`。

```
goose@goose:~/docker_image$ ll
```

```
总计 670156
```

```
drwxrwxr-x 2 goose goose 4096 1月 1 20:12 ./
drwxrwxr-x 6 goose goose 4096 12月 24 20:49 ../
-rw----- 1 goose goose 409648128 1月 1 20:12 backend.tar
-rw----- 1 goose goose 29795328 1月 1 20:12 frontend.tar
-rw----- 1 goose goose 246778880 1月 1 20:12 mysql.tar
-rw----- 1 goose goose 10245248 1月 1 20:12 redis.tar
```

5. 上传文件到服务器 (Upload)

我们需要将镜像文件和配置文件上传到服务器。使用 `scp` 命令（请替换 `user`、`your-server-ip` 和 `your-server-path` 为实际值）。

假设服务器项目目录为 `/project/ds`（如果不存在请先在服务器创建：`mkdir -p /project/ds`）。

1. 上传基础文件：

```
scp docker-compose.yml xxx.sql user@your-server-ip:/project/ds/
```

注意： 请检查你的 SQL 文件名。如果 `docker-compose.yml` 里是 `./scripts/init-mysql.sql`

- `./volumes/mysql:/var/lib/mysql` # 数据持久化
- `./scripts/init-mysql.sql:/docker-entrypoint-initdb.d/init.sql` # 初始化脚本

但若只有 `xxx.sql`，请在上传后在服务器上重命名，或修改 `docker-compose.yml`。

2. 上传镜像包（文件较大，耗时较长）：

```
scp backend.tar frontend.tar user@your-server-ip:/project/ds/
```

6. 服务器加载与部署 (Server Deploy)

登录到服务器进行最终部署。

1. SSH 登录服务器：

```
ssh user@your-server-ip  
cd /project/ds
```

2. 加载 Docker 镜像：

```
docker load -i backend.tar  
docker load -i frontend.tar
```

加载完成后，可以使用 `docker images` 查看是否出现了 `ds-backend` 和 `ds-frontend`。

3. 启动服务：

```
docker-compose up -d
```

4. 验证部署：

- 查看容器状态： `docker-compose ps`
- 查看日志： `docker-compose logs -f`
- 浏览器访问服务器 IP： `http://your-server-ip:80`